

Specification: Options Trading Support for TradingView Broker API

This document serves as an implementation guide for an AI developer agent expanding an existing **TradingView Broker REST API + WebSocket** integration from Equities to Options.

1. Executive Summary & Goals

Scope

Extend the existing, fully functional Equity Broker API backend to support single-leg Options trading.

Key Objectives

- * Support OCC/OSI options symbology resolution in `/symbol_info` and search endpoints.
- * Extend order endpoints (`/previewOrder`, `/placeOrder`) to process option actions (`buy_to_open`, `sell_to_close`, `sell_to_open`, `buy_to_close`) and respect contract multipliers (100).
- * Implement risk pre-checks for user options approval levels (Levels 1–4) and options buying power.
- * Update position and account streaming endpoints to deliver contract-level details and real-time options metrics.

2. Symbology & Contract Specification

Options symbols must follow the standard **Option Clearing Corporation (OCC) / Option Symbology Initiative (OSI)** format:

```text

Format: [UNDERLYING (6)] + [YYMMDD] + [C/P] + [PRICE (8 digits)]

```

- * **Example:** `AAPL260918C00150000`
- * **Underlying:** `AAPL`
- * **Expiration:** September 18, 2026
- * **Type:** Call (`C`)
- * **Strike:** `$150.00` (`00150000` = $\$150.00 * 1000$)

`GET /symbol_info?symbol=AAPL260918C00150000`

Request from TradingView

```
```http
GET /symbol_info?symbol=AAPL260918C00150000 HTTP/1.1
Host: api.yourbroker.com
Authorization: Bearer <user_oauth_token>
```
```

Expected Broker Response

```
```json
{
 "symbol": "AAPL260918C00150000",
 "ticker": "AAPL260918C00150000",
 "name": "AAPL Sep 18 2026 150 Call",
 "type": "option",
 "session": "0930-1600",
 "timezone": "America/New_York",
 "minmov": 1,
 "pricescale": 100,
 "unit_id": "contract",
 "multiplier": 100,
 "currency-code": "USD",
 "underlying-symbol": "AAPL",
 "option-type": "call",
 "strike": 150.00,
 "expiration": 1789689600
}
```
```

3. API Endpoints Specification

A. Accounts Endpoint (`GET /accounts`)

Request from TradingView

```
```http
GET /accounts HTTP/1.1
Host: api.yourbroker.com
Authorization: Bearer <user_oauth_token>
```
```

Expected Broker Response

```
```json
[
```

```
{
 "id": "ACC-88392",
 "name": "Options Trading Account",
 "currency": "USD",
 "equity": 50000.00,
 "buyingPower": 100000.00,
 "optionBuyingPower": 35000.00,
 "optionLevel": 3
}
]
...
```

---

#### ### B. Order Preview Endpoint (`POST /previewOrder`)

Calculates order impact, margin requirements, total estimated cost, and validates options level permissions prior to placement.

#### #### Formula for Total Cost calculation

```
```text
Total Premium = Order Price * Quantity * Multiplier (100)
```
```

#### #### Request from TradingView

```
```http
POST /previewOrder HTTP/1.1
Host: api.yourbroker.com
Authorization: Bearer <user_oauth_token>
Content-Type: application/json
```

```
{
  "accountId": "ACC-88392",
  "symbol": "AAPL260918C00150000",
  "side": "buy",
  "qty": 5,
  "type": "limit",
  "price": 4.50,
  "duration": {
    "type": "DAY"
  },
  "customFields": {
    "optionAction": "buy_to_open"
```

```
}  
}  
...
```

Expected Broker Response (Success)

```
```json  
{
 "confirmId": "PREV-994123",
 "symbol": "AAPL260918C00150000",
 "qty": 5,
 "side": "buy",
 "estimatedPrice": 4.50,
 "estimatedTotal": 2250.00,
 "commission": 3.25,
 "marginRequirement": 2250.00,
 "warnings": []
}
...
```

#### #### Expected Broker Response (Error: Option Level Exceeded)

```
```json  
{  
  "s": "error",  
  "errmsg": "Account ACC-88392 (Level 1) is not authorized for Sell To Open (Naked) option  
transactions. Level 3 required."  
}  
...
```

C. Place Order Endpoint (`POST /placeOrder`)

Places the single-leg options order into the execution broker's Order Management System (OMS).

Request from TradingView

```
```http  
POST /placeOrder HTTP/1.1
Host: api.yourbroker.com
Authorization: Bearer <user_oauth_token>
Content-Type: application/json
```

```
{
```

```

"accountId": "ACC-88392",
"symbol": "AAPL260918C00150000",
"side": "buy",
"qty": 5,
"type": "limit",
"price": 4.50,
"duration": {
 "type": "DAY"
},
"customFields": {
 "optionAction": "buy_to_open"
}
}
...

```

#### #### Expected Broker Response

```

```json
{
  "id": "ORD-1002934",
  "status": "working",
  "symbol": "AAPL260918C00150000",
  "qty": 5,
  "side": "buy",
  "price": 4.50,
  "type": "limit",
  "customFields": {
    "optionAction": "buy_to_open"
  }
}
...

```

D. Positions Endpoint (`GET /positions`)

Returns current open option positions with multiplier-aware market value and unrealized profit/loss.

Request from TradingView

```

```http
GET /positions?accountId=ACC-88392 HTTP/1.1
Host: api.yourbroker.com
Authorization: Bearer <user_oauth_token>

```

...

#### #### Expected Broker Response

```
```json
[
  {
    "id": "POS-OPT-001",
    "symbol": "AAPL260918C00150000",
    "qty": 5,
    "side": "buy",
    "avgPrice": 4.50,
    "currentPrice": 5.20,
    "marketValue": 2600.00,
    "unrealizedPI": 350.00,
    "customFields": {
      "underlying": "AAPL",
      "multiplier": 100,
      "strike": 150.00,
      "optionType": "call",
      "expiration": 1789689600
    }
  }
]
```
```

---

#### ## 4. Real-Time Streaming Updates (WebSocket)

The system pushes position, order status, and account metric updates through the existing WebSocket channel.

#### #### Order State Change Frame

```
```json
{
  "event": "order_update",
  "data": {
    "id": "ORD-1002934",
    "accountId": "ACC-88392",
    "symbol": "AAPL260918C00150000",
    "status": "filled",
    "filledQty": 5,
    "avgFillPrice": 4.48,

```

```

    "customFields": {
      "optionAction": "buy_to_open"
    },
    "updateTime": 1789689605
  }
}
...

```

Position State Change Frame

```

```json
{
 "event": "position_update",
 "data": {
 "id": "POS-OPT-001",
 "accountId": "ACC-88392",
 "symbol": "AAPL260918C00150000",
 "qty": 5,
 "avgPrice": 4.48,
 "currentPrice": 5.20,
 "unrealizedPI": 360.00,
 "marketValue": 2600.00
 }
}
...

```

---

#### ## 5. Risk Engine & Business Rules Matrix

The backend risk gateway must evaluate all inbound option orders against the user's risk profile before sending them to the execution layer.

Option Action	Required Level	Risk & Margin Check Rule
**Buy To Open (BTO)**	Level 1	Cash available $\geq$ (Limit Price * Qty * 100) + Fees
**Sell To Close (STC)**	Level 1	Existing long position quantity $\geq$ Order Qty
**Sell To Open (STO) - Covered Call**	Level 1	Long stock position $\geq$ (Order Qty * 100)
**Sell To Open (STO) - Cash Secured Put**	Level 2	Cash available $\geq$ (Strike Price * Order Qty * 100)
**Sell To Open (STO) - Naked Options**	Level 3 / 4	Reg-T or Portfolio Margin requirement satisfied
**Buy To Close (BTC)**	Level 1	Existing short option position quantity $\geq$ Order Qty

---

## ## 6. Implementation Task Checklist for Developer Agent

- [ ] **Data Model Extension**: Add fields for `multiplier`, `underlying`, `strike`, `expiration`, `optionType`, and `optionAction` to internal database models and validation schemas.
- [ ] **Symbology Parser**: Implement an OSI parser module to translate between standard OCC strings and internal contract identifiers.
- [ ] **/symbol\_info` Update**: Return correct metadata fields (`multiplier: 100`, `unit\_id: "contract", `type: "option"`) when an option symbol is queried.
- [ ] **Risk Gateway Module**: Build a middle layer function checking `optionLevel` and `optionBuyingPower` prior to executing `/previewOrder` or `/placeOrder`.
- [ ] **Order Multiplier Engine**: Ensure price and quantity calculations consistently apply the 100 contract multiplier across pricing, total cost, and PnL calculations.
- [ ] **Position & Account Adapters**: Map options positions and option buying power into standard TV REST API and WebSocket streaming frames.
- [ ] **Unit & Integration Tests**: Implement integration tests verifying BTO, STC, STO, and BTC execution flows, along with edge case handling for insufficient options buying power and unauthorized option level attempts.